

Lecture 7: The Krusell–Smith Method

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 19, 2026

University of Toronto

Krusell-Smith

Setup: Aiyagari with Aggregate TFP Shocks

Aggregate TFP $Z_t \in \{Z_{\text{bad}}, Z_{\text{good}}\}$ Markov with Π_Z .

Idiosyncratic state: (z_{it}, b_{it})

- Productivity $z_{it} \in \{z_{\text{lo}}, z_{\text{hi}}\}$ Markov with Π_z
- Wealth b_{it} .

Technology: Cobb–Douglas with TFP Z_t :

$$Y_t = Z_t K_t^\alpha L^{1-\alpha}.$$

L is assumed constant at the stationary mean of z .

Prices

Gross interest rate and wage given by Z_t, K_t, L ,

$$R(Z_t, K_t) = 1 + \alpha Z_t (K_t/L)^{\alpha-1} - \delta,$$

$$w(Z_t, K_t) = (1 - \alpha) Z_t (K_t/L)^\alpha.$$

Problem: Household price forecasts both **affect** and **depend on** future K_{t+1} .

K_{t+1} depends on Λ_t

$$K_{t+1} = \int b'_i(b_i, z_i; Z_t, K_t) d\Lambda_t(b_i, z_i).$$

Policy $b'_i(b_i, z_i; Z_t, K_t)$ depends on price forecasts.

Price R_{t+s}, w_{t+s} forecasts depend on Λ_{t+s} through K_{t+s} .

Note: Same K_t from different Λ_t can imply different K_{t+1} .

Bellman Equation

Distribution Λ becomes a state variable:

$$V(b_i, z_i; \Lambda, Z) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i; \Lambda', Z') \mid z_i, Z],$$

Λ is infinite-dimensional

Λ_t is a **distribution** over every (b, z) .

The “true” Λ_t is infinite-dimensional.

Even if we discretize Λ_t into a matrix, **every gridpoint is a state variable**.

With $N_b = 80$ wealth gridpoints and $\#z = 2$ productivity levels, grid has 160 points.

A V grid over possible Λ values: 5^{160-1} cells. About 10^{111} .

Tractably Parametrizing $V(\dots ; \dots, \Lambda)$

Two ideas:

- (a) **Reduce Λ .** (Krusell-Smith.) Use moments of Λ_t instead of the whole distribution.
- (b) **Reduce the representation.** (nVFI.) Parameterise $V(\Lambda, \cdot)$ by a flexible family of functions (e.g. a **neural network**).

Krusell–Smith Method

Microfoundation: Bounded Rationality

Krusell and Smith (1998): Assume the **household** only observes moments of Λ_t , not the whole distribution.

i.e. solve a model of $V(b, z; Z, K)$, not $V(b, z; Z, \Lambda)$.

The Approximate Law of Motion (ALM)

Assume household uses a forecast rule for K :

$$\log K_{t+1} = a_0(Z_t) + a_1(Z_t) \log K_t.$$

Two coefficients per Z state. For 2-state Z , four numbers total.

Log-linear. K-S find this is actually a very good fit.

The Bounded-Rationality Bellman

Replace Λ with K in the state:

$$V(b_i, z_i, Z, K) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i, Z', K')],$$

subject to

$$c_i + b'_i = R(Z, K) b_i + w(Z, K) z_i,$$

with the agent's *belief*

$$K' = \exp(a_0(Z) + a_1(Z) \log K),$$

and Z' drawn from Π_Z .

The Behavioural Chain (Five Stages)

Assuming the approximate law of motion (ALM), we can solve the model by simply adding two Stages to the household block:

1. $z_shock: z_{it} \rightarrow z_i^{t+1}$ (Markov on z).
2. $income: b'_i \leftarrow R b_i + w z_i$, with (R, w) from (Z, K) .
3. $savings$: pick b_i^{t+1} .
4. $K_evolve: K_t \rightarrow K_{t+1}$.
5. $Z_shock: Z_t \rightarrow Z_{t+1}$.

$$z_shock \circ income \circ savings \circ K_evolve \circ Z_shock.$$

Solving this chain gives us household **behaviour** given $a_0(\cdot)$, $a_1(\cdot)$, and allows us to simulate the **true** model forward.

K-S Outer Loop

1. **Pick** an initial ALM $(a_0^{(0)}, a_1^{(0)})$.
2. **Solve** the 4D household VFI given the current ALM.
3. **Simulate** a long economy under realised $\{Z_t\}$ and *true* aggregation $K_t = \int b d\Lambda_t$.
4. **Refit** (a_0, a_1) by OLS of $\log K_{t+1}$ on $\log K_t$, separately per Z .
5. **Iterate** until ALM coefficients stop moving.

Implementing Krusell–Smith

Parameters

Krusell-Smith calibration (1998):

- $\beta = 0.96$, $\gamma = 1$ (log utility), $\alpha = 0.36$, $\delta = 0.025$.
- $Z \in \{0.99, 1.01\}$ with Π_Z persistent (0.875 on-diagonal).
- $z \in \{0.07, 1.0\}$.

Grids (small for example):

- $N_b = 80$ on log-spaced $[0, 80]$.
- $N_K = 5$ on linear $[10.5, 14.5]$.
- Cell count: $80 \times 2 \times 2 \times 5 = 1600$.

State Layout

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(p.b_min, p.b_max; length=p.N_b, spacing=:log)),  
    StateAxis(:z,      p.z_grid),  
    StateAxis(:Z_idx,  [1, 2]),  
    StateAxis(:K,      continuous_grid(p.K_min, p.K_max; length=p.N_K)),  
)
```

- Two continuous axes (:wealth, :K), two categorical (:z, :Z_idx).

Stages 1–2: z_{shock} and income

Stage 1–2

```
z_shock = MarkovStage(layout; axis=:z, transition=pΠ._z)

income = WealthChangeStage(layout; wealth_post=(cell; env) -> begin
    Z = p.Z_vals[cell.Z_idx]
    (;R, w) = ks_prices(cell.K, Z, p)
    R * cell.wealth + w * cell.z
end)
```

Note. This reads (Z, K) from the cell coordinates.

Stage 3: ConsumptionSavings

Stage 3

```
savings = ConsumptionSavingsStage(layout;  $\beta = p\beta.$ ,  
  utility=(cell, c; env) -> u_crra(c, Val(py.)),  
  monotone_search=:divide_conquer)
```

- Divide-and-conquer search exploits monotonicity in wealth.

Stages 4–5: K_evolve and Z_shock

Stage 4–5

```
K_evolve = WealthChangeStage(layout; wealth_axis=:K,  
    wealth_post=(cell; env) ->  
        exp(env.a0[cell.Z_idx] + env.a1[cell.Z_idx] * log(cell.K)))  
  
Z_shock = MarkovStage(layout; axis=:Z_idx, transition=pΠ._Z)
```

Build Household Block

```
chain = z_shock ◦ income ◦ savings ◦ K_evolve ◦ Z_shock

hh = define_moments!(chain;
    K_supplied=at_end(integrand=:wealth, reduce=sum))
```

Inner solve: VFI Given $a_0(\cdot), a_1(\cdot)$

```
a0_init = [log(K_bar), log(K_bar)]  
a1_init = [0.0, 0.0]  
env0 = make_env(hh_4d; a0=a0_init, a1=a1_init)  
  
res4d = solve_steady_state_given_env!(hh_4d, env0; lambda_tol=1e-5, lambda_maxiter=50_000)
```

First guess. A *constant* ALM: $K_{t+1} = \bar{K}$ for both Z states ($a_1 \equiv 0$, $a_0 \equiv \log \bar{K}$).

Simulation Household Block

When we simulate we want *realised* (Z_t, K_t) , not the agent's belief.

The behavioural chain's `K_evolve` and `Z_shock` encode the *belief*, not the true law of motion for K .

The true chain has just the first three stages (`z_shock` `○` `income` `○` `savings`) and Z, K are updated in an **outer loop**.

Simulating the True Model

Each period:

1. Compute $K_t = \sum_{b,z} b \Lambda_t[b, z]$.
2. Lift Λ_t to a 4D distribution, adding (Z_t, K_t) as “idiosyncratic” state variables.
3. Simulate the 4D distribution using the behavioural chain (5 stages).
4. Marginalise out (Z, K) to get Λ_{t+1} on (b, z) .
5. Draw $Z_{t+1} \mid Z_t$ from Π_Z .

Refit the ALM by OLS

```
function regress_ALM(K_path, Z_idx_path, z_idx, window)
    rows = [t for t in window if Z_idx_path[t] == z_idx]
    x = log.(K_path[rows])
    y = log.(K_path[rows .+ 1])
    # OLS:  $y = a_0 + a_1 * x$  ;  $R^2 = 1 - SS_{res}/SS_{tot}$ 
    ...
    return (; a0, a1, R^2, n=length(rows))
end

fit_bad = regress_ALM(sim0.K_path, sim0.Z_idx_path, 1, window_fit)
fit_good = regress_ALM(sim0.K_path, sim0.Z_idx_path, 2, window_fit)
```

The Outer Loop

(Too long for the slide; see notebook.)

```
function ks_iterate(p::KSPParams, K_bar,  $\Lambda_0$ ; update_speed=0.5, rtol=1e-3, maxiter=20, T_sim
    =2000, T_burn=500)
    hh_4d = behavioural_alm_household(p)
    a0 = [log(K_bar), log(K_bar)]
    a1 = [0.0, 0.0]
    err, iters = Inf, 0
    while err >= rtol
        env = make_env(hh_4d; a0, a1)
        solve_steady_state_given_env!(hh_4d, env; lambda_tol=1e-5, lambda_maxiter=50_000)

        sim = ks_simulate(hh_4d, p; T=T_sim,  $\Lambda_{\text{init}}=\Lambda_0$ , Z_init=1)
        window_fit = (T_burn + 1):(T_sim - 1)
        fit_bad = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
        fit_good = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)

        a0_new = [fit_bad.a0, fit_good.a0]
        a1_new = [fit_bad.a1, fit_good.a1]
```